2026

How we made Trail of Bits AI-Native (so far)

Dan Guido, CEO | March 4, 2026 | [un]prompted

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

AI • PRODUCTIVITY

Thousands of CEOs just admitted AI had no impact on employment or productivity—and it has economists resurrecting a paradox from 40 years ago.

BY SASHA ROGELBERG | REPORTER

FEBRUARY 17, 2026, 1:32 PM ET

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

This article came out two weeks ago. Thousands of CEOs surveyed, and the result was: AI had no measurable impact on employment or productivity.

Economists are calling it the new Solow paradox. The original was Robert Solow in 1987 saying "you can see the computer age everywhere **but** in the productivity statistics." Same thing is happening with AI now.

Obviously, I don't think AI doesn't work. I think most companies are doing it wrong.

They gave people tools without changing the system. Everyone gets a ChatGPT license, and leadership waits for the productivity numbers to move. They don't.

That's the gap between AI-assisted and AI-native. One is a tool, the other is an operating system. The rest of this talk is about what it took to build the operating system.

What does AI-native mean?

'We use AI' vs 'AI is on the team'

AI-assisted	People use AI tools to do their existing jobs faster (chatbots, copilots, summarizers)
AI-augmented	Workflows are redesigned around AI capabilities (agents in the loop, AI-first processes)
AI-native	The organization is architected from the ground up assuming AI is a core participant in delivery, operations, and how expertise compounds

At ToB: AI-native means our security expertise compounds.
Every project makes the next one faster, and every engineer operates with an arsenal of specialized agents built from 14 years of knowledge.

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Before I talk about what we did, I want to define what we're even aiming for. Because "AI-native" gets thrown around a lot and it means different things to different people.

The way I think about it, there are three levels. Most companies are at level one.

- AI-assisted is where almost everyone starts. You give people access to ChatGPT. They use it to draft emails, generate boilerplate, summarize documents. It's a productivity tool. The org doesn't change. The workflows don't change. You just do the same things a little faster.
- AI-augmented is where you start redesigning workflows. You're not just using AI as a tool. You're putting agents in the loop, changing how work actually flows. Maybe the AI does the first pass on a code review and the human does the second. The process itself is different.
- AI-native is the structural shift. The org is designed from the ground up assuming AI is a core participant. Not a tool you pick up, but a teammate that's always there. Your knowledge management, your delivery model, your expertise, all designed to be consumed and amplified by agents.

At Trail of Bits, what this means concretely: our security expertise compounds as code. Every engagement we do, the skills and workflows we build make the next engagement faster. Every engineer operates with an arsenal of specialized agents built from 14 years of audit knowledge. That's not "we use AI." That's "AI is on the team."

What people are actually resisting

01 Self-enhancing bias. A model of you outperforms you. The more senior and successful you are, the harder this hits. You believe in the intuition that got you here.

02 Identity threat. Security researchers identify deeply with their craft. AI that replaces skill feels like an attack on who you are.

03 Opacity. People think they understand human judgment and don't understand AI. In reality, they understand neither, but the feeling of not understanding kills the feeling of control.

04 Intolerance for imperfection. We forgive our own mistakes but not the machine's. One bad AI output confirms the bias. (Dietvorst et al.: people abandon algorithms after seeing them err once.)

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

In studies of why people resist technology adoption, the core insight is: the problem is never the software. It's people's unwillingness to accept that something else might be better than their intuition.

Self-enhancing bias:

- Tendency to perceive oneself in an overly favorable manner. Driven by the need to maintain self-esteem, this bias causes individuals to take credit for success and blame external factors for failures
- In reality, Paul Meehl & Robyn Dawes research: take the variables an expert says they use, build even a crude linear model = the model outperforms the expert
- Not because the model is smarter. Because it's **consistent**. Humans work differently on different days, overfocus on one thing or another.
- Gets worse with seniority. You believe in the intuition that got you where you are.

Identity threat:

- Cooking study: same kitchen automation device framed two ways: "Does the cooking for you" vs "helps you cook better." Same device. People who identify as cooks rejected the first, accepted the second
- Another dimension to this: Imagine a Tattoo robot: people don't want a robot giving them a tattoo (because it’s symbolic, human craft). But they're fine with a robot removing one (instrumental, no symbolism).
- For us: security auditing is deeply symbolic work. Have to frame AI as "makes

- you a more dangerous auditor", not "does the audit"

Opacity:

- Medical AI study (Nature Human Behaviour 2021): subjective understanding of human judgment is high, AI judgment is low. But objective understanding of both is near zero. Virtually identical.
- People feel like they understand how a doctor diagnoses. They can't explain it either.
- Fix: comparative reasoning. Skin cancer AI that just says "suspicious" causes people to distrust it. Same AI that shows "why is this not benign?" with visual comparisons to benign moles -> trust equalizes.

Intolerance for imperfection:

- Dietvorst et al.: In a study, participants watched an algorithm outperform a human forecaster. But after seeing the algorithm make one error, they abandoned it and went back to the human — even though the human was worse
- Fix (follow-up paper): let people slightly modify the algorithm. Even one adjustable parameter was enough to overcome the aversion

The remedies that actually worked

Designing the system around the resistance, not against it

BARRIER	REMEDY	WHAT WE BUILT
Self-enhancing bias	Make it measurable: you can't argue you're already good enough	Maturity matrix with visible levels
Identity threat	Skill-allowing, not replacing: frame AI as making experts more dangerous	Skills repos, Maturity Matrix that rewards building (not just using), hackathons, not mandates
Intolerance for imperfection	Reduce the ways AI can fail embarrassingly, curate quality	Curated marketplace, sandboxing, guardrails, footgun reduction
Opacity	Write the rules so people know what's safe and approved	AI Handbook, usage policy, clear boundaries
(All of the above)	Make adoption visible and fast	Hackathons, copy-pasteable configs, **CEO goes first**

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Self-enhancing bias:

- The maturity matrix was controversial internally. Nobody likes being told they're at level 1.
- But that's the point: you can't argue you're already good enough when there's a visible ladder. It makes the conversation concrete instead of "I don't think AI is useful."
- Also creates social proof. When you see peers at level 2 or 3, the passive 50% starts moving.

Identity threat:

- This was the hardest one and the one we spent the most time on.
- The key insight: people accept skill-allowing automation but reject skill-replacing automation. Same tool, different framing.
- So we never asked anyone to stop being a security expert. We gave them a new way to express that identity.
- The skills repos are the clearest example. When a senior engineer writes a constant-time-analysis skill, they're not being replaced: they're becoming more permanent. Their expertise is encoded and reusable. That's an identity upgrade, not a threat.
- The maturity matrix reinforces this: level 3 isn't "uses AI the most." It's "invents new ways, builds tools." The identity of the expert shifts from "I don't need AI" to "I'm the one who makes the AI dangerous."
- Hackathons instead of mandates. Never forced adoption. Created a space where people could experiment on their own terms. Adoption felt like

- exploration, not compliance.

Intolerance for imperfection:

- If someone's first AI experience is bad, you've lost them for months. Dietvorst's research: one error and people abandon the algorithm.
- So we invested heavily in reducing the ways AI can fail embarrassingly. Curated marketplace means no random plugins with backdoors. Sandboxing means Claude Code can't accidentally delete your work. Guardrails and footgun reduction mean fewer "AI did something stupid" stories circulating in Slack.

Opacity:

- People didn't know what was safe to use, what was approved, what data they could put where. That uncertainty created paralysis.
- The AI Handbook made it concrete: here's what's approved, here's what's not, here are the exceptions, here's who to ask. Clear rules restored feeling of control.
- Remedy for opacity is comparative reasoning: don't just say "trust this," explain why. The handbook does that: it doesn't just list tools, it explains the risk model behind each decision.

Making adoption visible and fast:

- This one underpins everything else. Deferred benefits kill adoption. If setup takes an hour and the first result is mediocre, you've confirmed every skeptic's priors.
- Copy-pasteable configs, one-command setup, standardized toolchain — all designed so the first experience is fast and good.
- **CEO goes first matters** more than people think. The passive 50% watches what leadership actually does, not what it says. If the CEO is visibly using these tools every day, it signals this isn't a fad.

The “operating system” model

01 Pick a standard toolchain so you can support it.

02 Write the rules so risk conversations stop being ad hoc.

03 Create a capability ladder so improvement is expected, measurable, and rewarded.

04 Run tight adoption sprints so the org keeps pace with releases.

05 Package the learnings into reusable artifacts (repos, configs, sandboxes) so the system compounds.

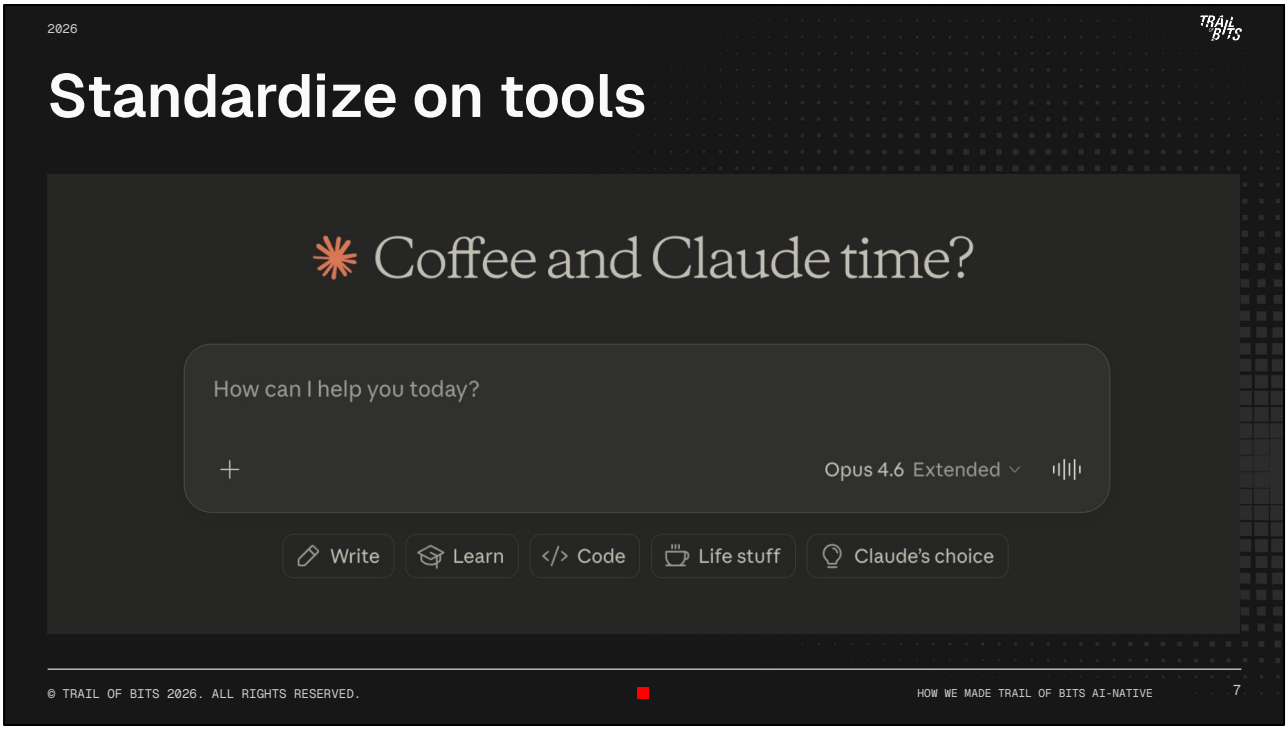
© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

So that's the problem and the approach. Now here's the actual system we built. Five parts, each one addresses the barriers we just talked about.

I'm going to walk through each of these in detail with screenshots and examples, but this is the overview. Think of it as a checklist: you need all five for it to compound.

Importantly, this isn't a strategy deck we wrote and handed to someone. We built every piece of this ourselves, open sourced most of it, and iterated on it in production with a 150-person company doing real client work.

Let's go through each one.



Step one was boring but critical: we standardized.

We got everyone on Claude Code, and we treat it like any other enterprise tool: supported configs, known-good defaults, and a clear path to ‘this is how we do it here.’

If you skip this step, you can’t build anything else. You end up with 40 different workflows and zero leverage

Write the rules

AI AT TRAIL OF BITS

This document provides comprehensive information about approved AI tools, services, and best practices for Trail of Bits team members. As AI capabilities continue to evolve rapidly, we've established guidelines to help you leverage these technologies effectively while maintaining our security and privacy standards.

To improve your chat prompts, take a short course in prompting from LearnPrompting.org or Anthropic, review the Prompt Engineering Guide, and read Anthropic's Claude Code Best Practices.

Join AI-Related Slack channels #ml-random #ml-experiments and #wg-ml-assurance.

There is a twice-monthly AI/ML Reading Group on the company Team Meetings calendar. The archive of papers, notes, and recordings is on Google Drive in the AI/ML Reading Group folder.

USAGE POLICY

All services listed in this document are approved for use with sensitive data, with these important exceptions:

• **Cursor** cannot be used for client code, except for blockchain client code. Use Claude Code or Continue.dev instead.

• **Meeting recorders** cannot be used for client meetings when we're hired by a lawyer under privilege.

The table below is a convenient lookup for various AI services and their pre-approved use for different teams/work. If you have any questions about any usage of any specific tool or service, ask in Slack or ask your manager.

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Next: we wrote an **AI Handbook**. Not to teach people how to prompt, but to remove ambiguity.
The key part is the **usage policy**: what tools are approved, and what isn't, especially for sensitive data.
For example, we explicitly call out things like:

- where a tool *can't* be used on client code,
- where meeting recorders are disallowed depending on engagement type,
- and where we want people to use a specific alternative.

This stops the endless Slack debate. It gives PMs and engineers a shared answer, and it makes client conversations consistent.

Once you have policy, you can safely push harder on adoption.

2026

TRAIL OF BITS

Make it measurable

AI MATURITY MATRIX

The specific tools and tasks below serve as examples and are by no means an exhaustive or requisite list for each role. Discuss how you use AI tools for tasks specific to your role with your manager to plot where you currently sit on the matrix.

ROLE	NOT ENGAGED (0) Not experimenting; no evidence of trying	CAPABLE (1) Experiments with tools personally	ADOPTIVE (2) Enthusiastic about new methods; actively shares the results with others	TRANSFORMATIVE (3) Invents new ways to use tools; maintains/builds tools
ASSURANCE	• Dismisses AI as hype or security risk • Refuses to use AI for any security work • Manually analyzes all vulnerabilities and writes reports from scratch • Believes AI can't understand security contexts	• Uses Claude for vulnerability explanations and report drafting • Uses Claude Code for code review, vulnerability analysis, and tool scripting • Uses Claude to summarize security papers and videos • Drafts blog posts using our Claude Project with Tech Clarity Expert style	• Maintains custom agent instructions (CLAUDE.md) • Contributes to Wilson, our internal AI auditing tool • Active in #mi-experiments, testing and verifying AI strategies • Uses NotebookLM to synthesize multiple audit reports	• Builds agent systems with Claude Agent SDK; runs agents that autonomously produce audit artifacts — findings, triage, report drafts • Develops systematic approaches to prompt optimization (e.g., DSPy, custom frameworks) • Designs workflows where agents execute full analysis passes, not just answer questions • Leads AI knowledge-sharing initiatives (Reading Group, workshops, team training) • Publishes blog posts on AI for security research
ENGINEERING	• Calls AI coding assistants "too risky" • Has never tested AI-generated code • Relies only on Stack Overflow snippets • Dismisses colleagues who use AI tools	• Uses Claude Code for code generation and explanation • Generates boilerplate with Claude Code • Can explain and debug code with AI assistance • Follows AI best practices from handbook	• Uses Claude Code subagents effectively • Maintains custom CLAUDE.md across projects • Skilled at writing project requirements docs and using planning mode • Contributes to building MCP servers	• Builds agent systems with Claude Agent SDK; runs autonomous agents that ship PRs, close issues, and benchmark results without hand-holding • Develops systematic approaches to prompt engineering (e.g., DSPy, evals) • Fine-tunes models or builds evals for security use cases • Develops AI-enhanced security tools (fuzzers, analyzers) • Designs agent workflows that produce multiple outputs from the same engineer
SALES	• Writes proposals and SOWs from old templates without AI input • Manually compiles weekly pipeline reports from HubSpot • Prepares for client calls by reading old emails • Tracks account activity and renewal timelines in spreadsheets	• Drafts proposals, SOWs, and client emails with Claude • Uses Claude to research prospects and prep for discovery calls • Summarizes meeting notes with Granola for call follow-ups and action items • Uses Claude to draft QBR decks and account review materials	• Creates Claude Projects for active deals with stakeholder maps and account context • Uses Claude Cowork with Granola, Slack, Gmail, and HubSpot MCP servers connected • Uses job-brand skills for competitive positioning and proposal differentiation • Builds reusable SOW and proposal templates that cut drafting time significantly	• Has agents producing recurring deliverables — pipeline reports, QBR prep, account reviews — autonomously • Creates AI-driven systems for conference prep: pre-booking meetings, enriching attendee lists, scoring leads • Builds tools the GTM team uses daily — SOW generators, competitive intel dashboards, capacity analyzers • Designs agent workflows that turn portfolio management from reactive deal-closing into systematic account growth
PROJECT MANAGEMENT & OPERATIONS	• Refuses to use AI for "sensitive" client work • Takes manual meeting notes only • Dismisses AI tools as making PMs lazy • Manually handles every ticket and task	• Drafts project status updates and client emails with Claude • Uses Granola for meeting notes and action item tracking • Summarizes client requirements and SOWs with Claude • Uses Claude to prep for kickoff calls and client check-ins	• Creates Claude Projects for active client engagements with project context • Uses Claude Cowork with Granola, Slack, and Gmail MCP servers connected • Automates QBR prep and status reporting with reusable prompts • Shares effective workflows and templates with the PM team	• Designs end-to-end AI-powered client lifecycle • Creates automated client reporting systems • Designs workflows where agents handle the information processing so PMs focus on client relationships • Has agents executing recurring operational work — status reports, QBR prep, client updates — autonomously

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

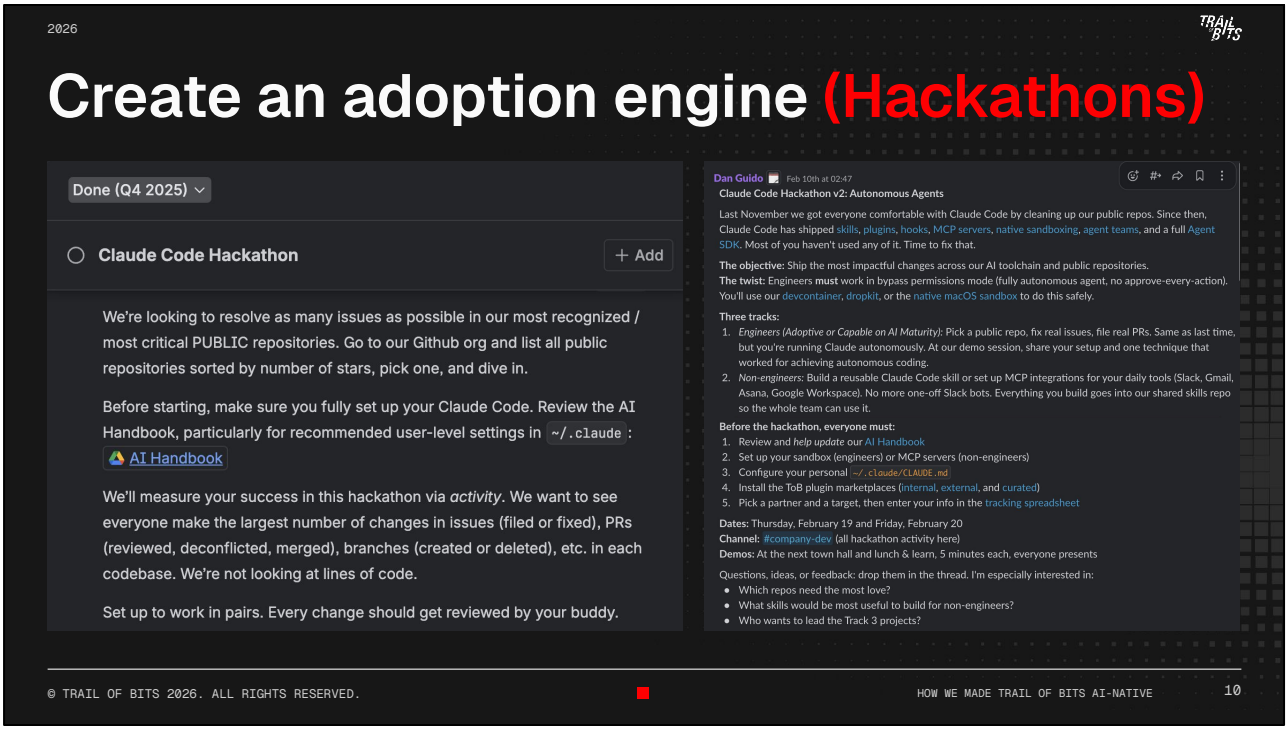
Then we built an **AI Maturity Matrix**.
You make AI usage a **first-class professional capability**, like 'can you use Git,' or 'can you write tests.'
It's not a vibe. It's a ladder:

- clear levels,
- clear expectations,
- a clear path up,
- and real consequences for staying stuck.

This is how you avoid two failure modes:

1. leadership wishing adoption into existence, and
2. the org splitting into 'AI people' and everyone else.

The matrix turns it into a normal part of performance, growth, and resource allocation



Now the fun part: **hackathons as a management system.** We run these short, focused sprints (2–3 days) with one objective. They're how we keep pace when the ecosystem changes every week.

Here's a kickoff message for one: **'Claude Code Hackathon v2: Autonomous Agents.'**
The two lines that matter are:

- **Objective:** ship the most impactful changes across our AI toolchain and public repos.
- **Twist:** engineers must work in **bypass permissions mode**. Meaning: fully autonomous agent, not approve-every-action.

That twist is intentional. It forces everyone to learn the real constraints: sandboxing, guardrails, and how to structure work so agents can succeed.

This second screenshot shows how we operationalize it. A few design choices here are important:

- We focus on **public repos** so we can move fast and show real outcomes.
- We measure success by **activity** (issues filed/fixed, PRs reviewed/merged, branches created/deleted)
- We have people work in **pairs**, and every change gets reviewed by a buddy.

So even the 'move fast' sprint has quality control built in.
Hackathons create motion. But motion doesn't compound unless you capture it.

Capture the work as reusable artifacts (Skills repos)

Available Plugins

Smart Contract Security

Plugin	Description
building-secure-contracts	Smart contract security toolkit with vulnerability scanners for 6 blockchains
entry-point-analyzer	Identify state-changing entry points in smart contracts for security auditing

Code Auditing

Plugin	Description
audit-context-building	Build deep architectural context through ultra-granular code analysis
burpsuite-project-parser	Search and extract data from Burp Suite project files
differential-review	Security-focused differential review of code changes with git history analysis
insecure-defaults	Detect insecure default configurations, hardcoded credentials, and fail-open security patterns
semgrep-rule-creator	Create and refine Semgrep rules for custom vulnerability detection
semgrep-rule-variant-creator	Port existing Semgrep rules to new target languages with test-driven validation
sharp-edges	Identify error-prone APIs, dangerous configurations, and footgun designs
static-analysis	Static analysis toolkit with CodeQL, Semgrep, and SARIF parsing
testing-handbook-skills	Skills from the Testing Handbook: fuzzers, static analysis, sanitizers, coverage
variant-analysis	Find similar vulnerabilities across codebases using pattern-based analysis

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Next: we turned learnings into **artifacts that scale**. The most important is a **skills repo**.
A ‘skill’ is not a chat transcript. It’s a reusable, structured workflow, ideally with examples, constraints, and a way to verify output.
We maintain:

- an **internal skills repo** for company-specific workflows, and
- an **external skills repo** so the broader community can validate and improve what we’re doing.
- (and a curated skills repo we’ll talk about next)

For a firm like ours, external validation matters: it keeps us honest, and it forces us to write things so other people can actually use them.

Safe third-party adoption (Curated marketplace)

README | CC-BY-SA-4.0 license | Security

Curated Skills Marketplace

Trail of Bits' reviewed and approved Claude Code plugins. Every skill and marketplace here has been vetted for quality and safety.

Why This Exists

We don't want people at Trail of Bits installing random plugins from GitHub repos we haven't reviewed. Published skills have been found with backdoors and malicious hooks, and the ecosystem has no built-in quality gate. This repo is how we solve that problem internally.

Everything here has been code-reviewed by Trail of Bits staff. We're sharing it publicly so the broader community benefits from the same vetting.

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Then we had another problem: once you tell people ‘go use skills and plugins,’ they’ll install random stuff.
So we created a **curated marketplace**: a ‘known good’ place for third-party skills.
This is basic enterprise thinking applied to agent tooling: if you want adoption, you need a safe supply chain.

Make defaults copy-pasteable (Claude Code configs)

Trail of Bits Claude Code Config

Opinionated defaults, documentation, and workflows for Claude Code at Trail of Bits. Covers sandboxing, permissions, hooks, skills, MCP servers, and usage patterns we've found effective across security audits, development, and research.

> Also see: skills · skills-curated · claude-code-devcontainer · dropkit

First-time setup:

```
git clone https://github.com/trailofbits/claude-code-config.git
cd claude-code-config
claude
```

Then inside the session, run `/trailofbits:config`. It walks you through installing each component, detects what you already have, and self-installs the command so future runs work from any directory. Run `/trailofbits:config` again after updates.

Contents

Getting Started

- Read These First
- Prerequisites
- Shell Setup

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Next we made the defaults dead simple.
We built a repo that centralizes **recommended Claude Code configuration**, so onboarding isn't 'tribal knowledge.'
This is where we put:

- known-good settings,
- recommended patterns for personal `~/.claude/CLAUDE.md`,
- and anything we want to standardize so people don't reinvent it.

Sandbox by default (Devcontainer + Dropkit)

dropkit

A command-line tool for managing DigitalOcean droplets with automated setup, SSH configuration, and lifecycle management.

Features

- 🚀 **Quick droplet creation** with cloud-init automation
- 🔑 **Automatic SSH configuration** - just run `ssh dropkit.<droplet-name>`
- 🔐 **Tailscale VPN** - secure access via Tailscale (enabled by default)
- 👤 **User management** - automatically creates your user account on droplets
- 🏷️ **Smart tagging** - organizes droplets by owner for easy filtering
- 🔒 **Security-first** - SSH key validation, confirmation prompts for destructive operations
- 📊 **Rich CLI** - beautiful tables, progress indicators, and helpful error messages
- 🔄 **Complete lifecycle** - create, list, resize, destroy droplets with ease
- 💤 **Hibernate/Wake** - snapshot and destroy to save costs, restore later with one command

Claude Code in a devcontainer

A sandboxed development environment for running Claude Code with `bypassPermissions` safely enabled. Built at Trail of Bits for security audit workflows.

Why Use This?

Running Claude with `bypassPermissions` on your host machine is risky—it can execute any command without confirmation. This devcontainer provides **filesystem isolation** so you get the productivity benefits of unrestricted Claude without risking your host system.

Designed for:

- **Security audits**: Review client code without risking your host
- **Untrusted repositories**: Explore unknown codebases safely
- **Experimental work**: Let Claude modify code freely in isolation
- **Multi-repo engagements**: Work on multiple related repositories

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Now the real constraint: if you want **autonomous agents**, you need **sandboxing**. We give people multiple safe lanes:

- a devcontainer option, and
- the native macOS sandboxing option,
- And Dropkit

The point isn't that everyone uses the same sandbox. The point is: everyone has a *safe* sandbox, and it's easy to adopt.

Reduce footguns (package managers)

developer-tools

Security hardening for developer machines.

configure-package-manager-defaults.sh

Deploys secure defaults for npm, pnpm, pip, Go, and Ruby to protect against supply chain attacks.

What it configures:

Setting	Location	Effect
`ignore-scripts=true`	Global npmrc (detected dynamically)	Blocks postinstall scripts (main malware vector)
`save-exact=true`	Global npmrc	Pins exact versions, prevents surprise upgrades
`prefer-binary=true`	`/Library/Application Support/pip/pip.conf`	Prefers wheels over source distributions
`GOPROXY=proxy.golang.org,off`	Per-user via `go env -w`	Forces proxy, blocks direct VCS fetches
`GOSUMDB=sum.golang.org`	Per-user via `go env -w`	Verifies checksums against transparency log
`gem: --no-document`	System gemrc (detected dynamically)	Skips doc generation, enforces HTTPS

Note: Config paths are detected dynamically. npm uses `npm config get globalconfig` (Homebrew: `/opt/homebrew/etc/npmrc`). Ruby uses `gem environment` (macOS: `/Library/Ruby/Site/gemrc`). pnpm reads the same npmrc file.

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Then we tackled the ‘death by papercuts’ risks.
One examples:
We hardened defaults through MDM, for example, rolling out more secure package manager defaults via Jamf because the easiest way to reduce risk is to make ‘the safe path’ the default path.”

We force the most secure defaults for all the standard package managers. Yes, this means ignore-scripts=true, but it also means we force a 7 day cooldown policy on all dependencies.

Connect with real tools (slither-mcp)

Slither MCP Server

Tests python 3.11+ License AGPL v3

A Model Context Protocol (MCP) server that provides static analysis capabilities for Solidity smart contracts using Slither.

Overview

This MCP server wraps Slither static analysis functionality, making it accessible through the Model Context Protocol. It can analyze Solidity projects (Foundry, Hardhat, etc.) and generate comprehensive metadata about contracts, functions, inheritance hierarchies, and more.

You can also use Slither MCP as an easy-to-use Slither API for other use cases.

Features

- **Caching**: Slither runs are cached to `{$PROJECT_PATH}/artifacts/project_facts.json` for faster subsequent loads
- **MCP Tools**: Query contract and function information through MCP tools
- **Security Analysis**: Run Slither detectors and access results with filtering
- **Comprehensive Analysis**: Extracts detailed information about:
  - Contract metadata (abstract, interface, library flags)
  - Function signatures and modifiers
  - Inheritance hierarchies
  - Function call relationships (internal, external, library calls)
  - Security vulnerabilities and code quality issues
  - Source code locations

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Finally, once you have policy, guardrails, sandboxes, and skills you can connect agents to real tools.
One example we've published is an MCP server for Slither.
Even if you don't care about Slither specifically, the point is: MCP turns your internal tools into something agents can use reliably, and your org can govern.
Ref Stripe Minions

Results (so far)

What changed after building the system

Tooling scale (across internal + public repos):

- 94 plugins, 201 skills, 84 agents, 29 commands, 125 scripts
- 414 reference files encoding domain expertise
- Covers the full consulting lifecycle: sales, proposals, audit setup, 20+ vulnerability-class-specific analyzers, fuzzing, report writing, delivery, project management, recruiting, onboarding

Delivery impact:

- For certain engagements: from ~15 bugs/week to ~200 bugs/week
- ~20% of all bugs reported to clients are now initially discovered by AI
- Sales team averaging $8M rev per rep (industry benchmark: $2-4M)

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Let me give you some numbers on what this system actually produced.

Across our internal and public skills repos, we have 94 plugins containing 201 skills, 84 specialized agents, and over 400 reference files encoding domain expertise. That's the compounding effect. Every engagement, every auditor, every experiment adds to the arsenal.

What's important is the breadth. This isn't just an auditing toolkit. We have skills for writing sales proposals, tracking project hours, onboarding new hires, prepping conference blog posts, and delivering government contract reports. The internal repo has 20+ plugins targeting specific vulnerability classes. Each one packages expertise that used to live in someone's head into something any auditor can invoke.

On the delivery side: for certain clients where the codebase and scope allow it, we went from finding about 15 bugs a week to 200. That's not a faster human. That's an auditor running a fleet of specialized agents doing targeted analysis across an entire codebase in parallel, then validating the results.

About 20% of all bugs we report to clients are now initially discovered by AI in some form. These aren't junk findings: they go into real client reports. An auditor still validates every one. But the AI is surfacing things humans would have missed or wouldn't have had time to look for.

On the business side: our sales team averages $7-8M in revenue per rep. The

consulting industry benchmark is $2-4M.

AI doesn't just help the engineers. The sales team has skills for proposal drafting, competitive positioning, conference prep, lead enrichment. The same system, same repos, same compounding effect.

And this is maybe 1 year into building the system seriously. The models are getting better every month. The skills repo grows every week. A year from now these numbers look very different.

The replicable recipe

01 Standardize on one agent workflow you can support.

02 Write an AI Handbook so risk decisions aren't ad hoc.

03 Create a capability ladder so improvement is expected.

04 Run short adoption sprints that force hands-on usage.

05 Capture everything as reusable artifacts: skills + configs + curated supply chain

06 Make autonomy safe with sandboxing + guardrails + hardened defaults

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Let me close with the essential starter path, the minimum viable 'AI-native org design.
1, 2, 3, 4, 5, 6
That's what we've done so far and it's already changed how fast we can ship and how quickly we can adapt.
If you want to copy this, copy the system, not the specific tools.

Open questions

What we haven't solved yet

1. Private inference: Can open models replace closed ones?
   a. Local models (Qwen3-Coder-Next) are interesting but not good enough
   b. We're evaluating on-prem inference servers to run 230B+ models at full precision
   c. Speed matters more than size. No one uses a slow model, even if it's smart.
   d. In the meantime: private inference providers like Tinfoil.sh are getting compelling.
2. Prompt injection and client code protection
   a. How do we prevent agents from exfilling client code? We're lucky so many ToB clients are OSS
   b. Today: Our primary internal audit tools can't touch the web
   c. Next (maybe): agent-native shells like nono and agentsh that enforce policy at the syscall level
3. Policy enforcement and continuous learning at scale
   a. How to turn the company's usage into a feedback loop that continuously improves the OS
   b. Next (maybe): a master MCP server with every internal resource that enforces policy

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

Things we're actively working on and don't have great answers for yet. If you're going through this process, you'll hit these too.

Private inference:

- Want local models for cost and confidentiality, but open models aren't good enough yet — 17% gap vs Opus on coding benchmarks
- Buying a $54K box (384GB VRAM) to run 230B+ models at full precision
- Key insight: speed drives adoption more than capability. Nobody uses a slow model.
- Private inference providers like Tinfoil (confidential computing on NVIDIA GPUs, cryptographically verifiable) are getting compelling. We've audited several.

Prompt injection:

- Existential question for using AI on client code. Problem: The data the agent works on is inherently accessible to it.
- Today: blunt instruments. Sensitive clients = no web access. Wilson controls network tightly.
- Next: agent-native shells (nono, agentsh) that enforce policy at the kernel level

Policy enforcement and continuous learning:

- We push settings via MDM, but we're not yet pulling signal back
- Turns the whole company into a feedback loop that improves the operating system weekly

- Long-term policy architecture: master MCP server between agents and internal resources, enforcing policy server-side (the Stripe model). Not there yet.

Finally: "bill for hours spent looking" vs "bill for expertise that directs the looking."

References

USE OUR GITHUB REPOS!

- trailofbits/skills
- trailofbits/skills-curated
- trailofbits/claude-code-config
- trailofbits/claude-code-devcontainer
- trailofbits/dropkit
- trailofbits/slither-mcp

TWEET AT US!

- @dguido + @trailofbits
- Email: dan@trailofbits.com

www.trailofbits/careers – come work with us

© TRAIL OF BITS 2026. ALL RIGHTS RESERVED.

10 new skills out YESTERDAY: fp-check, let-fate-decide, agentic-actions-validator, seatbelt-sandboxer

Things I didn't mention:

- Structured internal reports to mine it for vuln info
- Instrument weAudit for flight recorder logs as audits happen

Specific jobs open:

- Want an engineer to work directly with me on accelerating things from this talk
- Head of application security, lead a team of ~15 exceptionally overperforming consultants